

大模型提示词工程分类

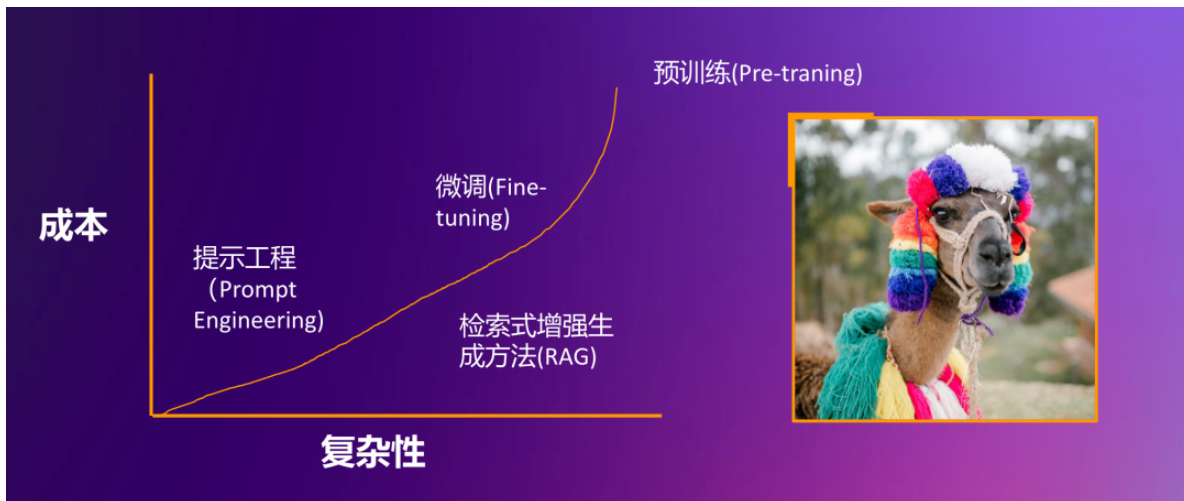
一、概述

预训练、微调和提示词工程是大型语言模型（LLM）生成质量优化的三种关键手段，各自具有不同的特点和优势。

预训练是 LLM 发展的基石，通过在大规模无标注语料库上进行自监督学习，赋予模型通用的语言理解和生成能力。**这种自然语言的通用知识为后续的微调和提示词工程奠定了坚实基础。**

微调则是利用有标注的任务数据，对预训练模型进行进一步调整和优化，使其更好地适应特定的下游任务。**微调的优势在于可以充分利用监督信号，提高模型在特定领域的生成质量。**但同时也存在过度微调导致过拟合、灾难性遗忘等潜在风险。

与之不同，提示词工程无需对模型参数进行更新，而是通过设计合理的提示，激发预训练模型中蕴含的知识，从而在下游任务中获得良好的生成质量。**这种方式更加高效灵活，避免了微调可能带来的问题。**但提示词的设计质量对模型表现影响重大，需要更多研究来探索通用有效的提示设计范式。



接下来，本文将重点讨论提示词工程技术。文章将介绍提示词工程的**基本原理、设计方法**，以及在不同任务场景下的**应用实践**，旨在为读者提供全面的理解和借鉴。同时也会分析提示词工程当前面临的挑战和发展方向，为后续研究指明路径。

二、提示词工程的分类

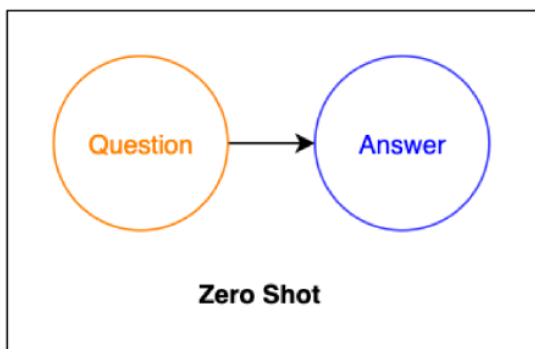
提示词工程的分类方法很多，本文将根据提示词工程技术发展演进路径，将类型分为：直接提示（Prompt）、链式提示（Chain）、图谱提示（Graph）、生成类提示（Generate）、集成式提示（Integrate）五种。

1、直接提示（Prompt）

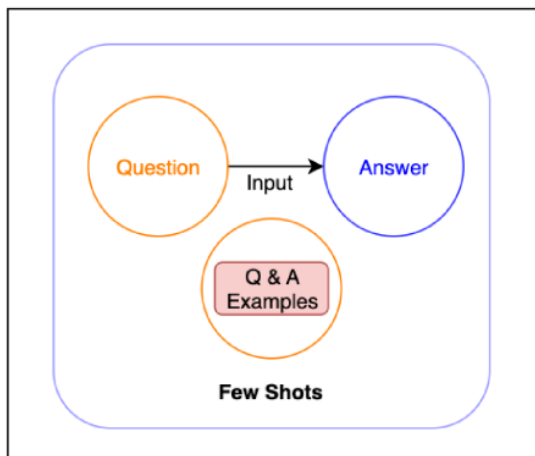
这种直接的提示词的方式侧重点在于通过特定的指令设计，来提示模型产生更好的输出。主要方法包括：

Zero-shot（零次提示）、Few shots（少量提示）、Act、ReAct 和 Directional Stimulus Prompting（方向性刺激提示）。

下面举几个例子，可以帮助大家加深这些方法差别的理解。

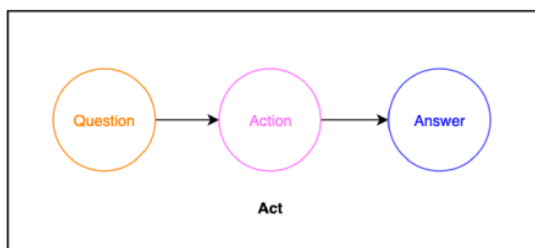


我该怎么表白？



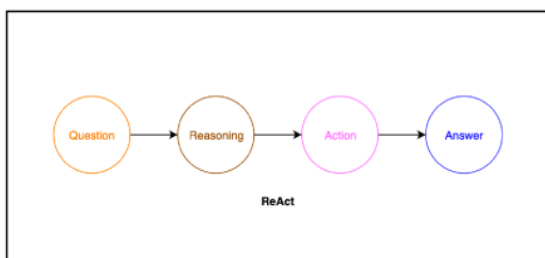
这是几个例子，如何模仿这些例子对她表白？

```
<example>
.....
</example>
<example>
.....
</example>
```



她喜欢哲学，
我该怎么在表白
的时候引入相关内容？

Zero Shot、Few Shots 和 Act 这三种偏向直接在问题中精心构造输入的内容。



初始提示词：
假如我们在圣诞节相遇，
该如何把这个场景融入表白中？
反应式提示：
如果她接受了我的表白，
下一步该怎么办？

而 ReAct 这种方式，更加关注外部的信息反馈，根据外部信息反馈后做归因然后再进行响应。

Directional Stimulus Prompting，是指在提示词中加入一些提示，以限制在特定领域或限定的方向上生成的内容。请看如下例子：

Input text

Article: (CNN) For the first time in eight years, a TV legend returned to doing what he does best. Contestants told to "come on down!" on the April 1 edition of **"The Price Is Right"** encountered not host Drew Carey but another familiar face in charge of the proceedings. Instead, there was **Bob Barker**, who hosted the TV game show for 35 years before stepping down in **2007**. Looking spry at **91**, Barker handled the first price-guessing game of the show, the classic "Lucky Seven," before turning hosting duties over to Carey, who finished up. Despite being away from the show for most of the past eight years, Barker didn't seem to miss a beat.

Reference

Bob Barker returned to host **"The Price Is Right"** on Wednesday . Barker, **91**, had retired as host in **2007** .

Directional Stimulus Prompting

Q: Summarize the above article briefly in 2-3 sentences **based on the hint**.
 Hint: **Bob Barker**; **TV**; **April 1**; **"The Price Is Right"**; **2007**; **91**.

Model output

A: On **April 1**, **Bob Barker** returned to the **TV** show **"The Price Is Right"** after eight years . Looking spry at **91**, Barker handled the first price-guessing game . Barker stepped down as host in **2007**, but didn't seem to miss a beat .

ROUGE-1: **48.39** 👍

Standard Prompting

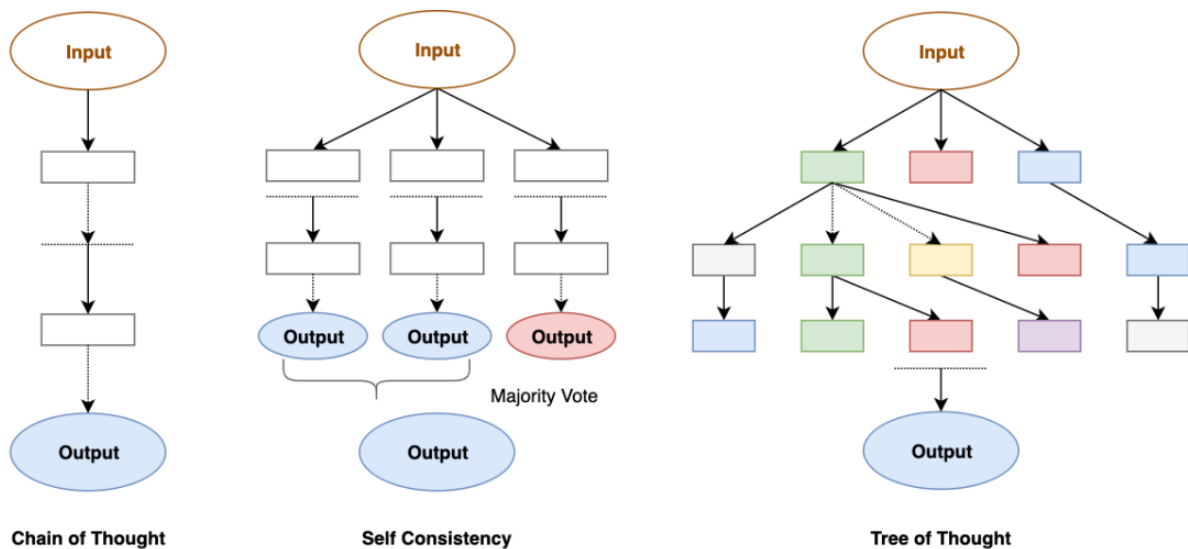
Q: Summarize the above article briefly in 2-3 sentences.

Model output

A: Bob Barker, the former host of \"The Price Is Right\", returned to the show for the first time in 8 years. He hosted the first game of the show, and then handed the hosting duties over to Drew Carey.

ROUGE-1: **34.48** 👎

2、链式提示 (Chain)



链式提示这种方式关注 LLM 的内部逻辑，比如思维链（Chain of Thought）、多模态思维链、思维树（Tree of Thought）。同时还关注在这个过程中的自治（Self-consistent）和自我反思（Reflexion）。

在这个过程中，一般会将复杂问题进行拆解，分成多个步骤进行推理。这样做的好处是，可以让 LLM 在推理过程中更容易获得正确的答案。

例如，对于一些早期的大语言模型，我们输入“我有 23 个苹果，吃了 20 个后又买了 6 个，现在还有几个？”，大概率会给出错误答案。但是 Amazon Bedrock 中大语言模型的响应，将问题分解成了多个步骤，然后逐步推理获得了正确答案。



我有23个苹果，吃了20个后又买了6个，现在还有几个？

2024/6/7 12:38:18



好的,让我们一步一步来算:

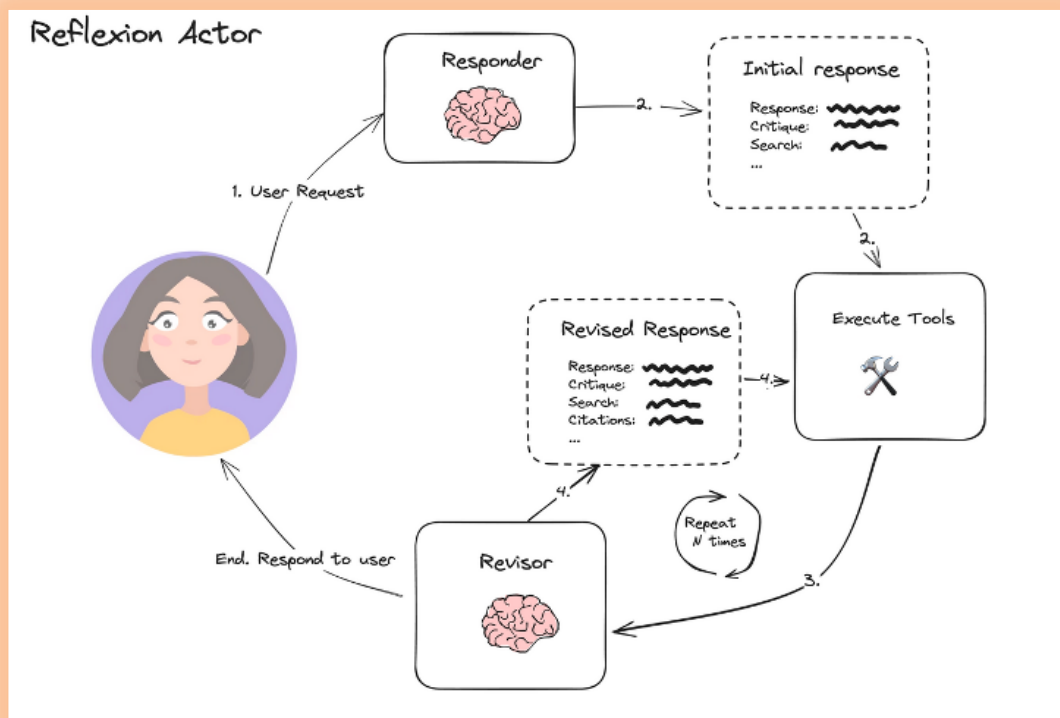
1. 一开始你有 23 个苹果。
2. 你吃了 20 个苹果,所以现在还有 $23 - 20 = 3$ 个苹果。
3. 然后你又买了 6 个苹果,所以现在总共有 $3 + 6 = 9$ 个苹果。

因此,现在你还有 9 个苹果。

2024/6/7 12:38:18

这种技术的核心就是在提示过程中表现为顺序的，可解释的，能够清晰地看到思维推理的过程。

Reflexion 顾名思义，是大语言模型对自己生成内容进行反思的一种过程。这种自我反思，会记录本轮的问题和输出的答案并作为记忆（上下文），然后再用大语言模型验证生成内容是否符合“初心”。**如果不匹配我们就会让这个过程的重复“持续思考”，直到获得理想的答案。**



下面看一个综合运用了多种链式范式综合案例。

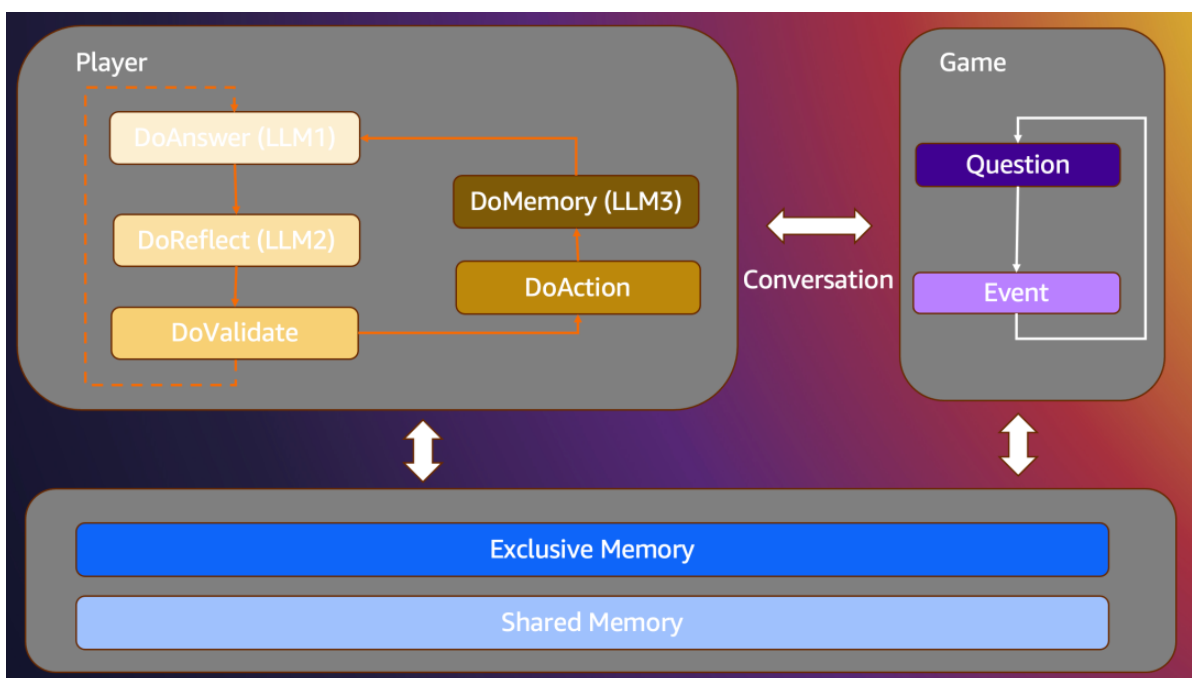


基于 Amazon Bedrock 上 LLM 的多 Agent 狼人杀游戏

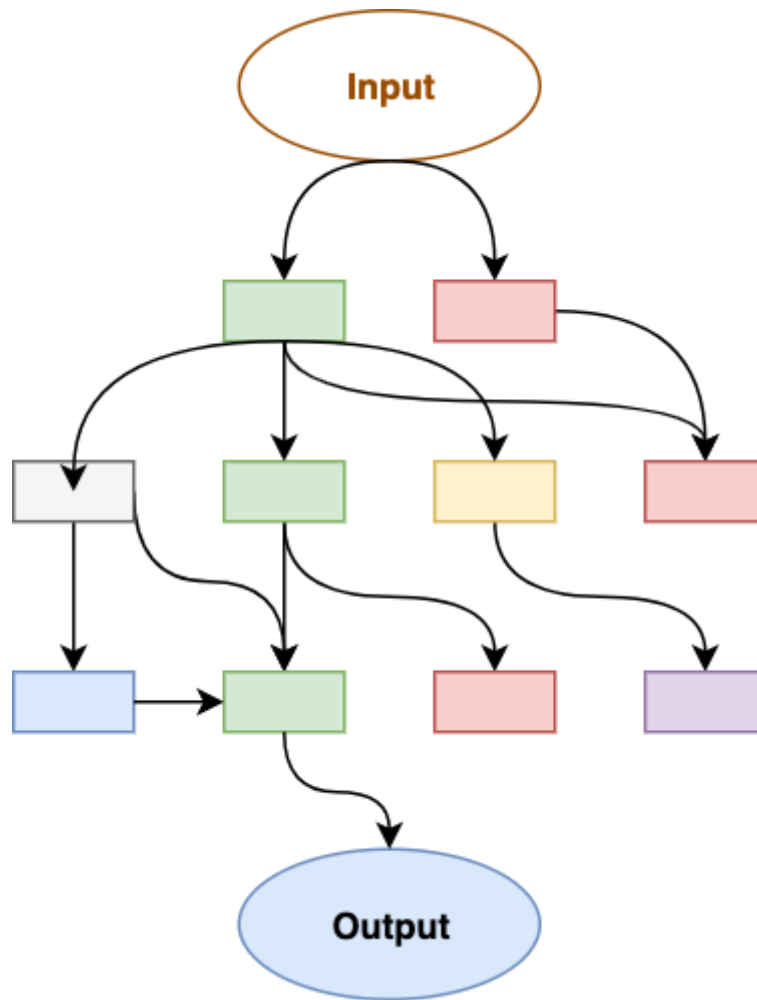
这是一个基于多 Agent 的人工智能狼人杀游戏，在这个游戏中，多个游戏角色由 LLM 来扮演，主要通过 Amazon Bedrock 中的大语言模型作为推理和发言的主模型，使用 Llama3 8B Instruct 和 Mistral 7B Instruct 作为 Reflexion 和总结性文字的输出。

不同游戏角色拥有不同的个性和记忆，角色会根据自己的记忆和游戏的流程不断去推理谁是地方阵营，并发动投票处决敌方。在每个轮次中，角色会对自己的发言进行自我反省，确定自己的发言是否对自己和己方阵营有利。这样做的好处就是可以避免角色出“昏招”。**由于这个反省模型和发言模型不一样，这样也可以避开由于单一模型推理过程中造成的“偏见”的问题。**

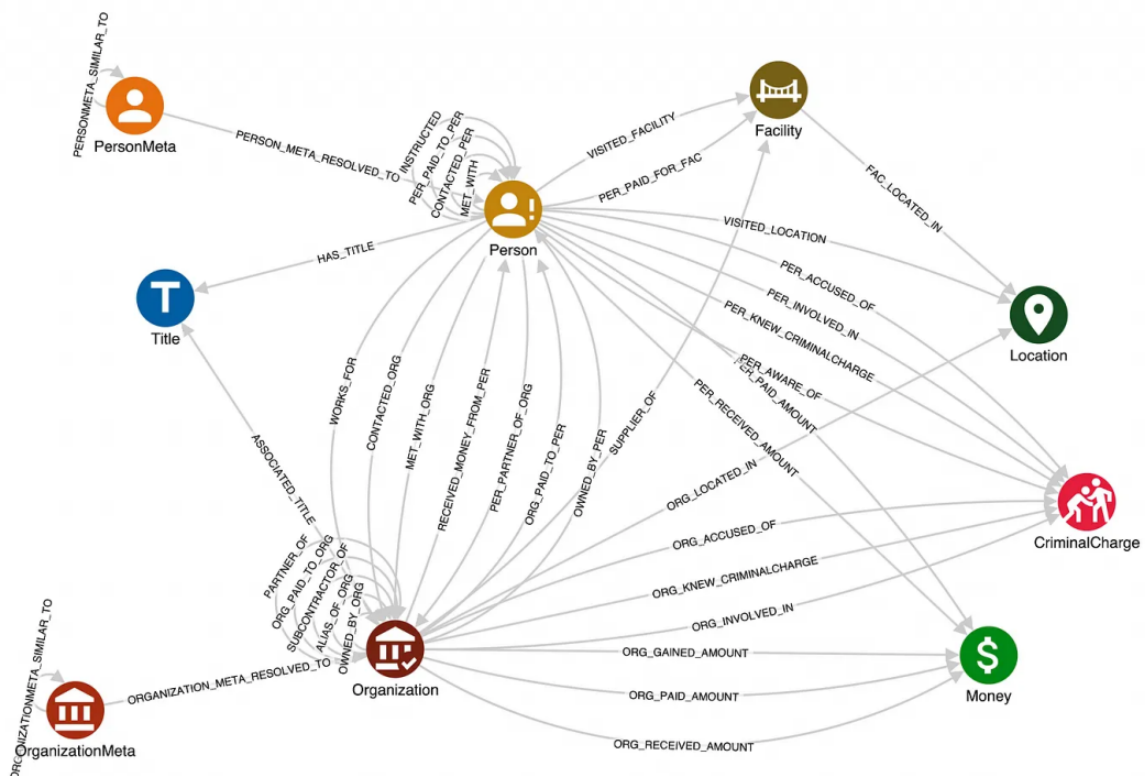
在整个游戏过程中，每个角色都会根据外部的的问题和事件，根据自己的独有记忆和共享记忆来对外部刺激作出响应。然后将“2 个大脑的声音”进行合理性验证，并得作出动作。这个游戏参考了“斯坦福小镇”的设计思路，对于制作开放式角色具有重要参考作用。



3、图谱提示 (Graph)



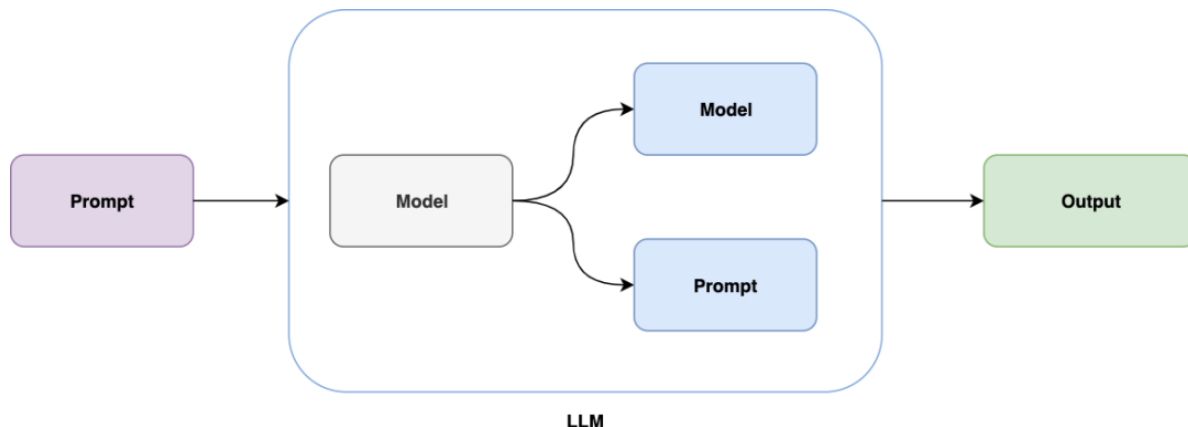
图谱提示和链条提示方式非常相似，都是关注大模型内部结构化的推理过程。但这个过程并不是线性的，而是假定训练过程中隐式地构建了网状或者是图形结构的知识图谱。在用户交互过程中，通过寻找不同知识的连接性和关联性，产生多个维度和更加复杂的非线性答案。



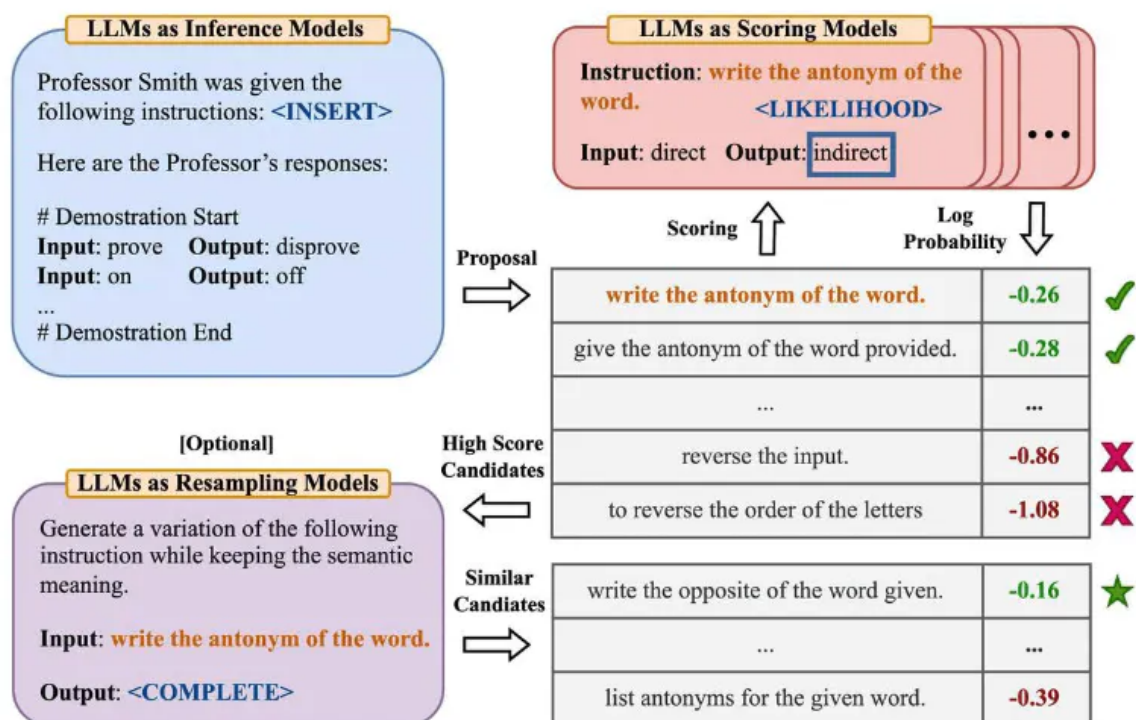
在这个范式下我们一般会先通过实体的识别，先找出相关问题的出发点。而这个出发点可能不止一个，然后通过实体关系寻找可能的下一个相关节点的相关信息。然后重复这个过程直到在所有关系或指定的有限关系中找到相关内容，并对问题给出正确的解。**这个范式适用于复杂数学问题、人物关系问题、犯罪线索发现等领域。**

4、生成类提示 (Generate)

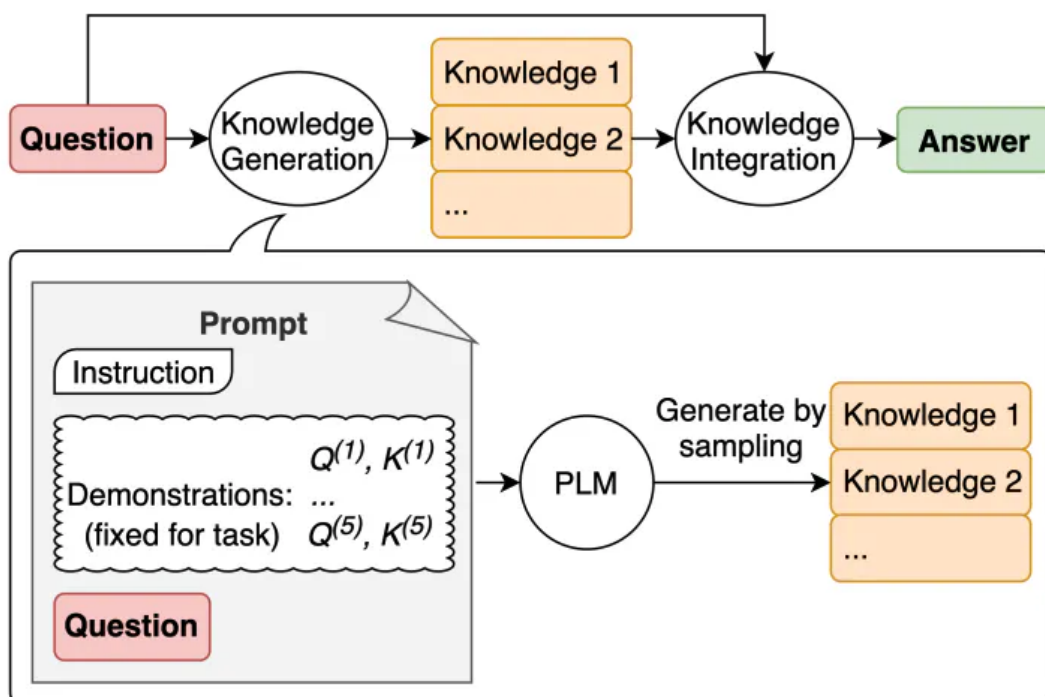
生成类提示，主要技术包括 Automatic Prompt Engineer 和 Generate Knowledge Prompting。



Automatic Prompt Engineer 关注自动化。这个方法让指令生成问题被构建为自然语言合成问题，使用 LLM 作为黑盒优化问题的解决方案来生成和搜索候选解。第一步选择一个语言模型用作推理，该模型接收输出数据以生成任务的指令候选项。这些候选解将指导搜索过程。使用大语言模型作为打分模型，然后根据计算的评估分数选择最合适的指令。



Generate Knowledge Prompting 关注内容生成。这种技术的特点是利用语言模型已有的知识进行与问题相关的知识片段的生成，以帮助模型做出更准确的预测。



无论是哪种技术，都是利用 LLM 内部的知识来迭代补充知识的生成。下面举个例子大家可以对比着看。

高尔夫的一部份就是试图获得比其他人更高的总分。



Bot

...

你说得对！高尔夫的目标是在尽可能少的杆数内完成球洞。这意味着你要在每杆击球中都尽力打出最好的成绩，并努力降低你的总杆数。当然，高尔夫也包含了其他一些方面，比如体育精神和享受户外活动。但总的来说，目标是尽可能打出最好的成绩。

使用某个大语言模型直接生成结果获得的错误例子

当然这个问题，现在使用更加强大的模型已经能直接获得正确答案。以下是使用 Amazon Bedrock 上 Amazon Bedrock 中的大语言模型的回答：



高尔夫的一部份就是试图获得比其他人更高的总分。

2024/6/7 13:27:00



你说得有点反了,高尔夫球的目标是尽可能获得较低的总分。

在高尔夫球比赛中,选手们需要在球场的每个洞中尽量用最少的杆次将球打入洞内。杆次越少,该洞的分数就越低。最终,所有洞的分数相加,总分最低的选手获胜。

所以高尔夫并不是追求较高的总分,而是努力降低总杆次、获得较低的总分。这需要选手精准控制球的力度和方向,并具备出色的策略和专注力。总的来说,高尔夫球是一项追求较低总分的运动。

2024/6/7 13:27:00

虽然这个例子中的问题, Amazon Bedrock 中的大语言模型已经能正确回答。**但是在很多场景下我们还是可以通过 Generate Knowledge Prompting 技术来提高问题解答的正确率。**下面我们看看这一切是如何发生的:

输入:

问题: 高尔夫球的一部分是试图获得比其他人更高的得分。是或否? 知识: 高尔夫球的目标是以最少的杆数打完一组洞。一轮高尔夫球比赛通常包括 18 个洞。每个洞在标准高尔夫球场上轮只打一次。每个杆计为一分, 总杆数用于确定比赛的获胜者。解释和答案:

输出:

不是, 高尔夫球的目标不是获得比其他人更高的得分。相反, 目标是以最少的杆数打完一组洞。总杆数用于确定比赛的获胜者, 而不是总得分。

在实际实施的过程中, 我们可能也会碰到一些输出不稳定的情况。具体情况需要根据业务来分析, 可以考虑综合运用多种 PE 技术手段进行优化。

5、集成式提示 (Integrate)

总的来说，提示词工程是一个充满活力和创新的领域，为大型语言模型的应用开辟了广阔的前景。通过不断探索和优化提示词的设计方法，我们能够最大限度地发挥模型的潜能，提高生成质量和任务适用性。未来，提示词工程必将与其他人工智能技术相互融合，为构建更加智能、高效和人性化的人机交互系统重要力量。

全套AI大模型资料

1. AI大模型学习路线图
2. AI大模型学习书籍
3. AI大模型视频教程
4. AI大模型项目实战
5. AI大模型面试题及答案解析
6. AI产品经理学习资料



微信扫码 免费领取